

Mobile & web experience specification

Screen mockups, user journeys, and system architecture for the technician mobile app — plus a walked-through screen flow for the web dashboard, from depot setup to live task execution and back into the mobile app.



PREPARED SEPTEMBER 11, 2026

COMPANION TO 00-08 IMPLEMENTATION GUIDES

Contents

Part 1 — Mobile app

Design system & principles

System architecture

User journeys (3)

Offline sync workflow

Screen mockups (8)

Part 2 — Web app

Sample screen flow, setup → live execution

Integration with the mobile app

Reference

Cross-references to guides 00-08

Design system

A field tool first: this palette and type system are chosen for outdoor legibility and unambiguous risk signaling, not decoration. Full rationale is in the frontend-design brief this spec follows.

Color

**Graphite 800**

#1B1F27

App background

**Graphite 700**

#242B35

Card surface

**Signal blue**

#3E7CB1

Primary action, links

**Risk — high**

#E5484D

High-risk chip, alerts

**Risk — medium**

#F2A93B

Medium risk, offline state

**Risk — low**

#3DAA6E

Low risk, synced state

Type

IBM Plex Sans carries all interface text — a technical, engineering-oriented family that reads clearly at small sizes on a phone screen in direct sunlight. **IBM Plex Mono** is used only for technical data that benefits from fixed-width alignment: equipment IDs, timestamps, sensor values, risk scores — never for UI labels or decoration.

Bearing vibration exceeds threshold

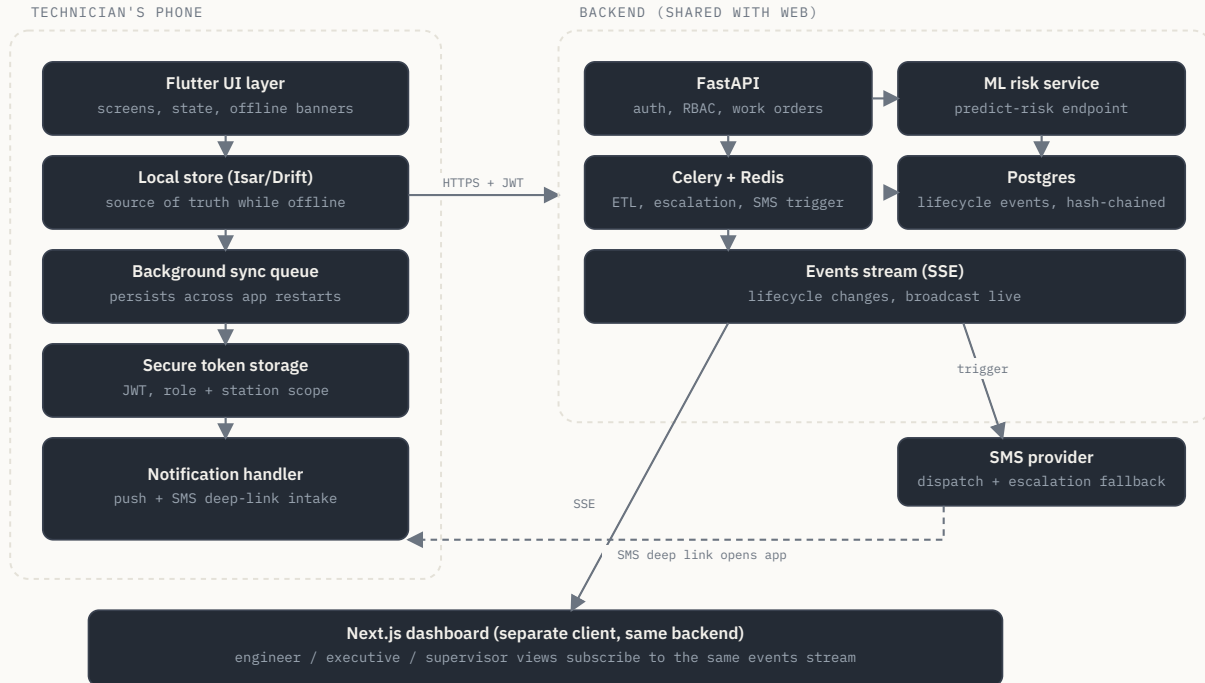
PUMP · P-14 · STN-B · risk score 0.87

Principles

- **Connectivity state is always visible, never hidden.** The synced/offline badge sits in the same place on every screen.
- **Risk color-coding is functional, not decorative** — the same three colors mean the same three things everywhere in the app.
- **Every screen assumes the network might not be there.** Nothing blocks on a live call that could instead queue locally.
- **The model explains itself.** Risk drivers are shown as plain-language reasons, never a bare score.

System architecture

The technician app's client-side layers, and where they meet the same backend contract the web dashboard uses. Full endpoint list is in 08-MOBILE-GUIDE.md §5.



Only the sync-queue and token layers change behavior when offline — everything above the HTTPS boundary is the exact same contract the web dashboard uses.

User journeys

Three journeys covering the same work order under three different connectivity/organizational conditions — together they define what the offline sync workflow (next page) actually has to handle.

JOURNEY A — HAPPY PATH

Alert to closed work order, fully connected

The baseline flow when the technician has signal the whole time — every other journey is a variation on where this one is interrupted.

- 1 Telemetry crosses the risk threshold**
The ML service scores incoming sensor data; risk_level reaches HIGH for pump P-14.
- 2 Engineer approves on the web dashboard**
Station engineer sees the risk drivers, checks parts availability, approves within minutes.
- 3 Technician is notified**
Push notification opens the app directly to the work order; SMS goes out in parallel as a fallback.
- 4 Technician reviews and accepts**
Risk drivers, reserved parts, and station location are all visible before they leave for the equipment.
- 5 Work is performed, steps tracked**
Each checklist step is marked complete in real time; status shows IN_PROGRESS on both mobile and web.
- 6 Completion recorded**
Parts used, notes, and a photo are submitted together as the closing record.
- 7 Audit trail and dashboards update**
The lifecycle event is hash-chained into the audit log; downtime window closes; executive numbers update.

JOURNEY B — OFFLINE FIELD VISIT

Same work order, zero signal at the equipment

Loading arms and pumps sit exactly where depot connectivity is worst — this is the common case, not the edge case.

- 1 Technician accepts while still connected**
Work order, risk drivers, and parts info are cached locally before they walk out of signal range.
- 2 Signal drops at the equipment**
The app switches its badge to OFFLINE; nothing about the visible work order changes.
- 3 Steps and notes are completed offline**
Every checklist tap, note, and photo queues locally instead of blocking on a network call.
- 4 Technician walks back into signal range**
The sync queue detects connectivity and starts pushing queued actions automatically.
- 5 Conflict check runs before pushing**
The app fetches current server-side lifecycle state first, confirms nothing changed, then pushes.
- 6 Queue clears, confirmation shown**
The technician sees a brief synced confirmation; nothing required them to notice or intervene.

JOURNEY C — ESCALATION WHILE OFFLINE

The work order moves without the technician

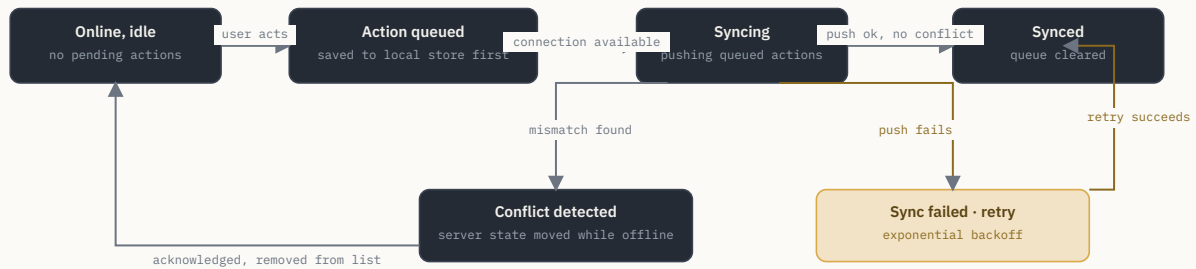
This is the case the sync-conflict screen exists for — connectivity gaps and organizational escalation can legitimately overlap.

- 1 Technician accepts, then loses signal for an extended stretch**
Phone goes into a low-signal basement area for a separate, unrelated job.
- 2 Approval SLA quietly breaches on a different, still-pending item**
Unrelated to this technician: a separate work order they hadn't picked up yet times out.
- 3 Supervisor reassigns the original work order**
Acting on the escalation, the supervisor reassigns P-14's work order to a second technician.
- 4 First technician regains signal**
Their sync queue tries to push a queued step update against a work order that has moved on.
- 5 Conflict surfaced, not overwritten**
The app shows exactly what happened and who has it now; the technician's notes are preserved on the record.
- 6 Technician acknowledges**
The item is removed from their active list with one tap; no data was lost on either side.

Offline sync workflow

The state machine behind Journeys B and C. Two branches exist specifically so a technician is only ever interrupted for a decision that genuinely needs one.

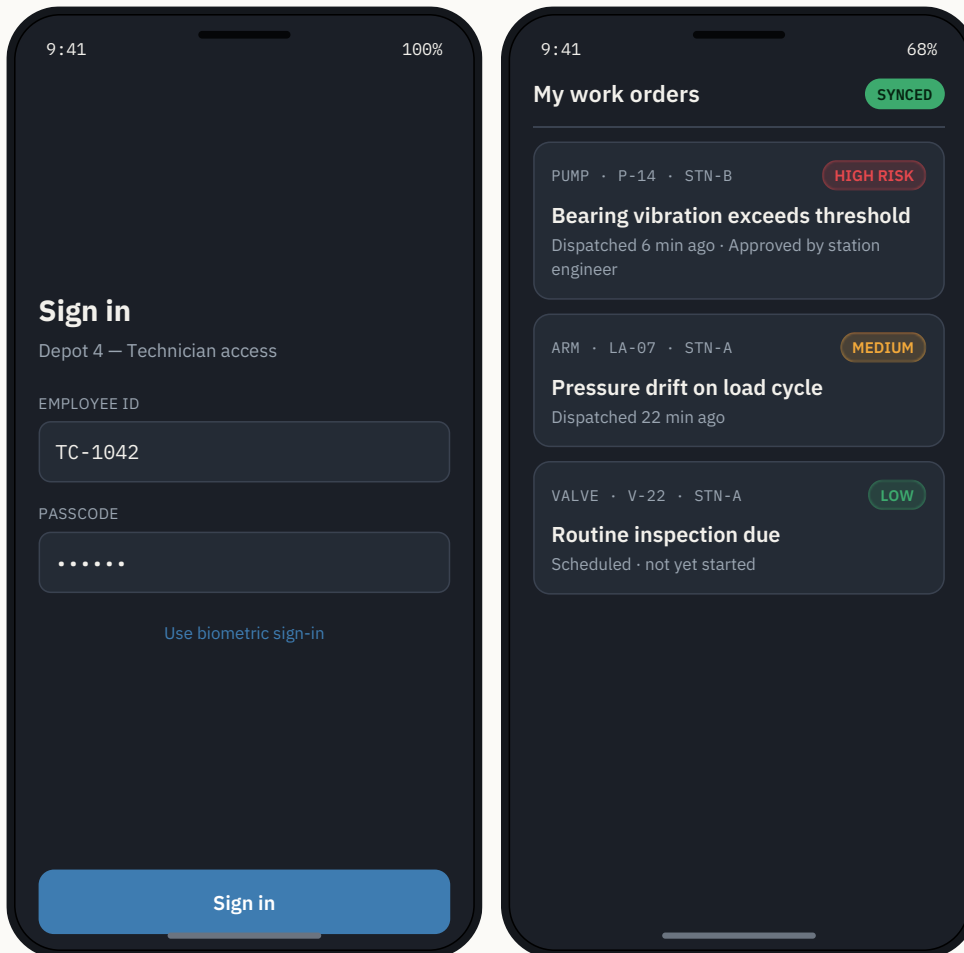
MOBILE SYNC QUEUE – STATE MACHINE



Nothing here requires the technician to notice or act — the two branches below only surface to the UI when a conflict needs a human decision; a plain retry stays silent.

Screen mockups

Eight core screens covering sign-in through completion, plus the two states unique to an offline-first field app.



1 Sign in

Employee ID + passcode, with biometric as a faster repeat-visit option. Kept deliberately short — this is often used one-handed, in gloves, in bright sun.

2 My work orders

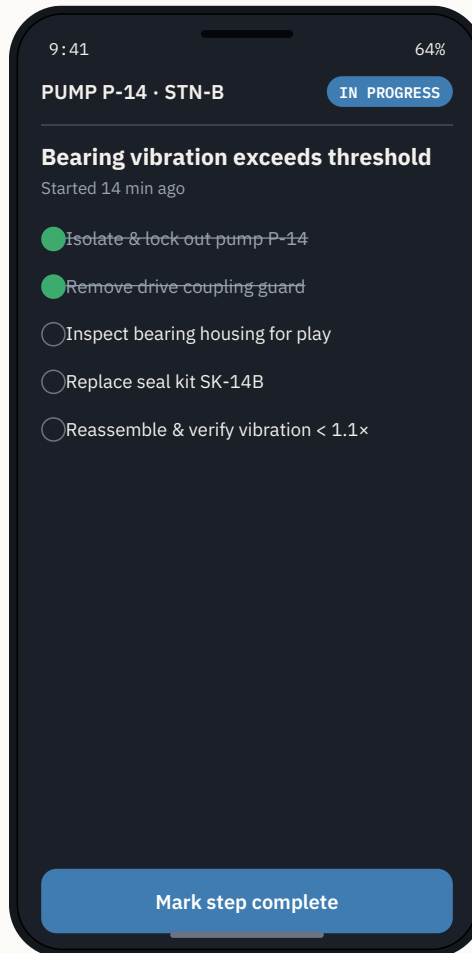
The home screen. A synced/offline badge is always visible in the same spot so connectivity state is never a mystery. Risk chips use the same color code everywhere in the app.

Screen mockups (continued)



3 Work order detail

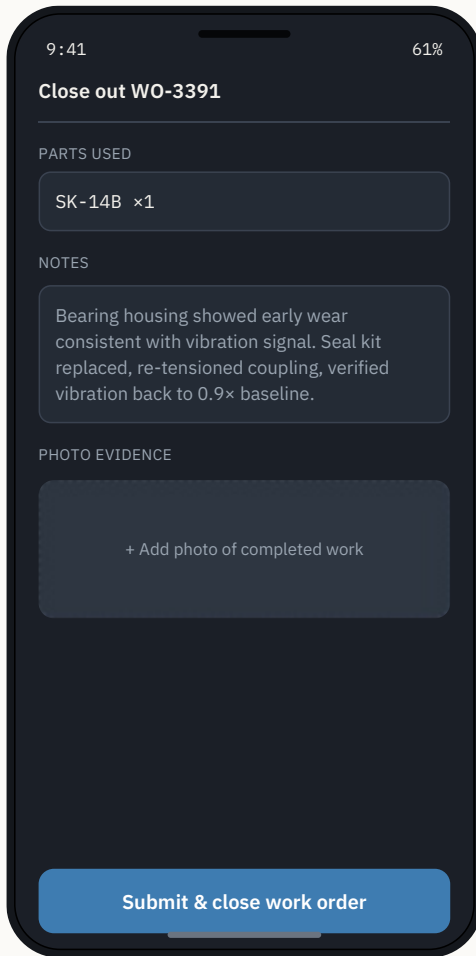
Risk drivers come straight from the ML model's `top_features` output, in plain language — not a score the technician has to interpret. Parts are already reserved before this screen ever loads.



4 Active task

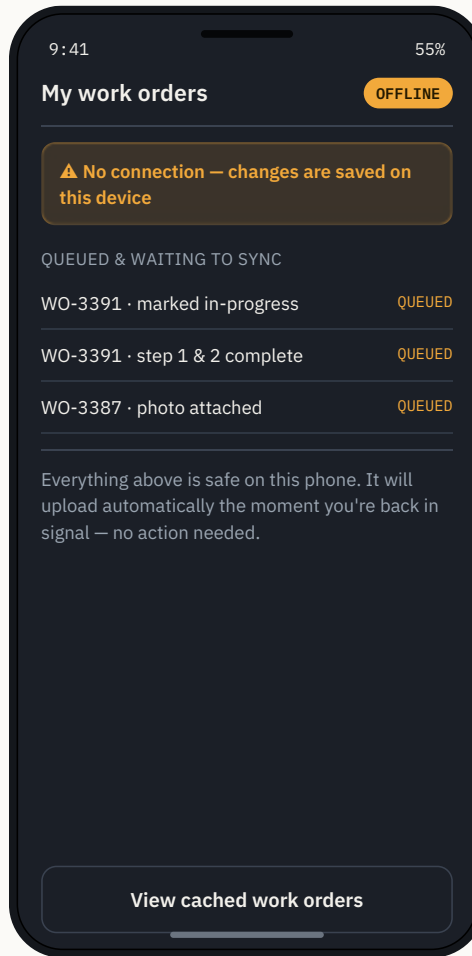
A step checklist keeps the technician oriented without requiring connectivity — every tap here is queued locally, not blocked on a network call.

Screen mockups (continued)



5 Close out

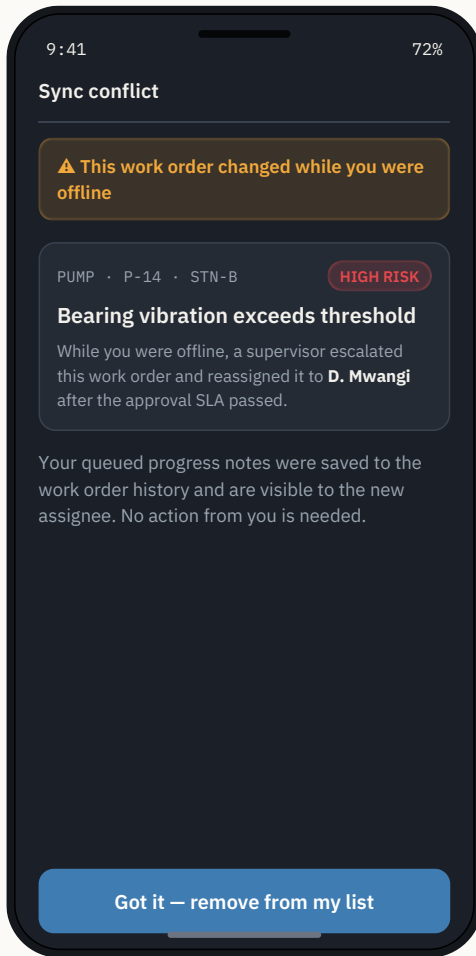
Notes, parts used, and a photo are captured together at completion — this is the record that becomes the audit-log entry and, in aggregate, the executive dashboard's numbers.



6 Offline mode

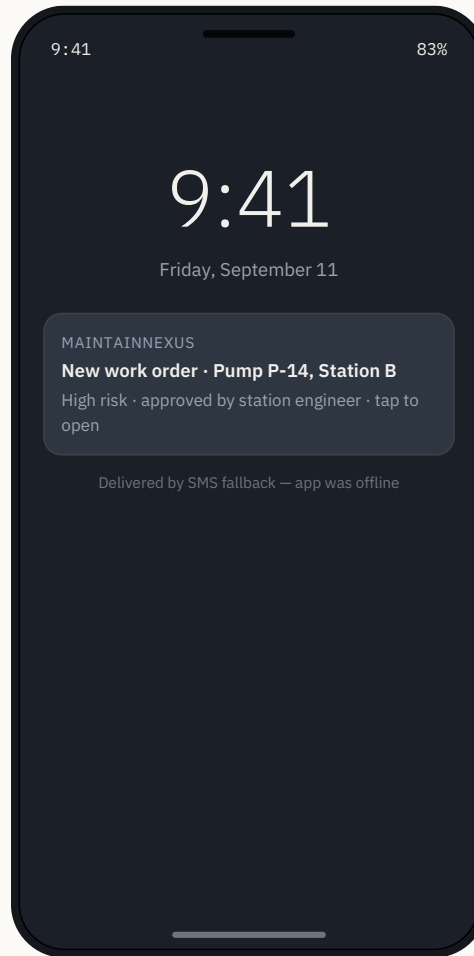
Offline is the expected state, not an error state. The queue is shown, not hidden, so the technician trusts that nothing is lost.

Screen mockups (continued)



7 Sync conflict

If the work order moved server-side while offline (reassignment, escalation), the technician is told plainly — never a silent overwrite in either direction.

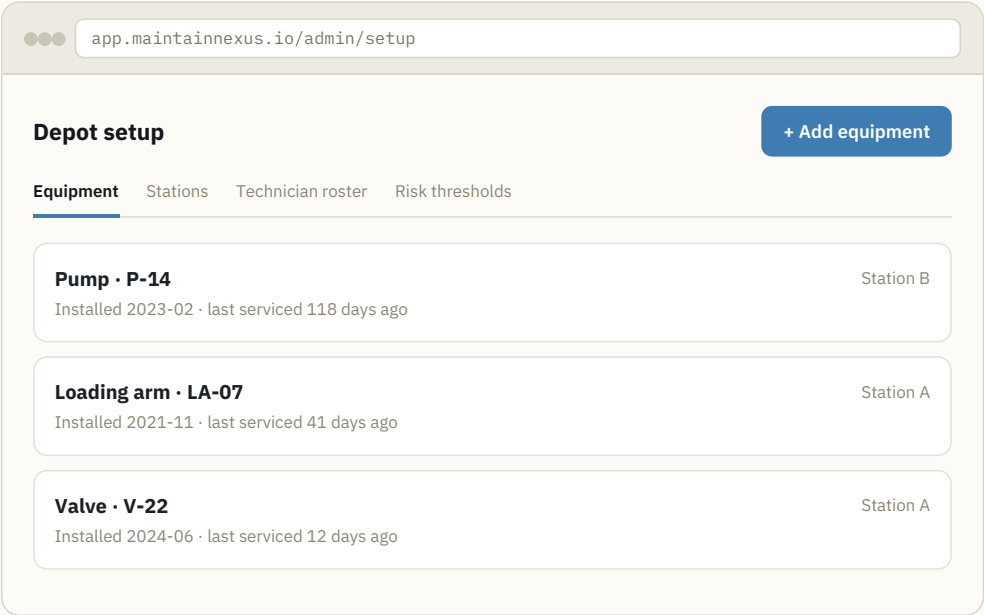


8 SMS fallback notification

When the app itself can't reach the network, the dispatch notice still arrives by SMS with a deep link, so a technician's first sign of a job never depends on data signal.

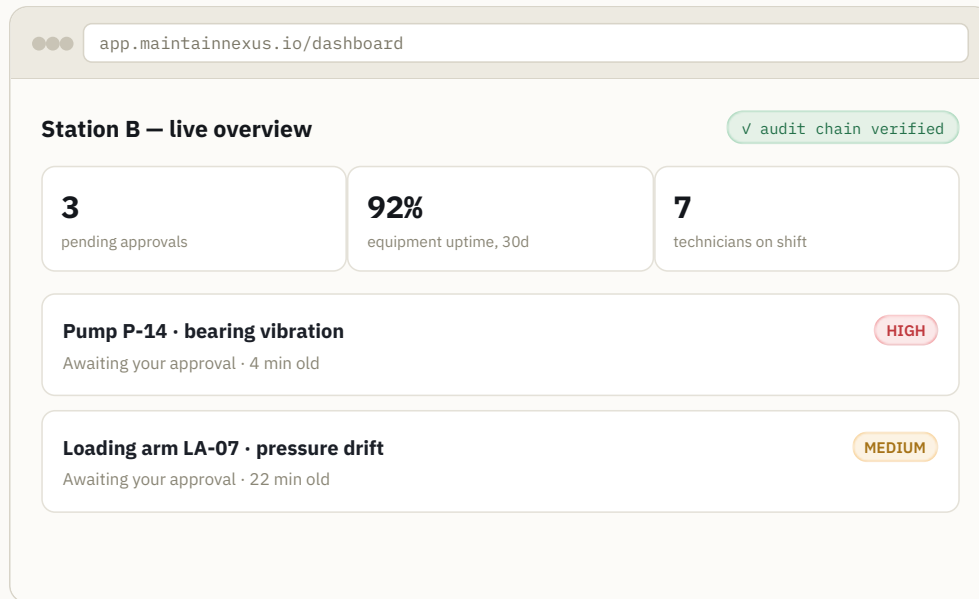
Sample screen flow: setup to live execution

A single walk-through, screen by screen, from an admin registering equipment through an engineer approving a work order to watching it close out — the same lifecycle event log the mobile app writes to.



1 Depot setup

An admin registers equipment, stations, the technician roster, and per-equipment risk thresholds before anything else runs. This is the one screen with no mobile counterpart — it's configuration, not day-to-day use.



2 Station dashboard

An engineer logs in and lands here: pending approvals for their station, plus a live uptime/staffing snapshot. The audit-chain badge in the corner is a standing integrity check, not a one-time confirmation.

Approval to dispatch

●●● app.maintainnexus.io/work-orders/3391

WO-3391 · pending approval

Pump P-14, Station B — bearing vibration exceeds threshold

WHY THIS WAS FLAGGED · risk score 0.87

Vibration	3.2× baseline
Pressure trend (6 hr)	Rising
Time since last service	118 days

Parts: seal kit SK-14B reserved, Bay 3 · Technician available: 2 certified on shift

Reject

Approve & dispatch

3 Pending approval detail

The same risk-driver breakdown the technician will eventually see on mobile (Mockup 3) — one model output, rendered for two audiences.

app.maintainnexus.io/work-orders/3391

WO-3391 · dispatched

v logged

● Created from telemetry alert

09:14 · ML risk service, score 0.87

● Approved

09:18 · you

● Technician assigned

09:18 · auto-matched, certified & on shift

● Dispatched — notified by push + SMS

09:19 · awaiting technician response

4 Dispatch confirmation

The moment of handoff to mobile: technician auto-matched, parts reserved, notification sent by push and SMS.

Live execution to close-out

●●●

app.maintainnexus.io/work-orders/3391

WO-3391 · live status

IN PROGRESS

●

Dispatched

09:19

●

Accepted by technician

09:24

●

Step 1 & 2 complete

09:41 · pushed the moment signal returned

●

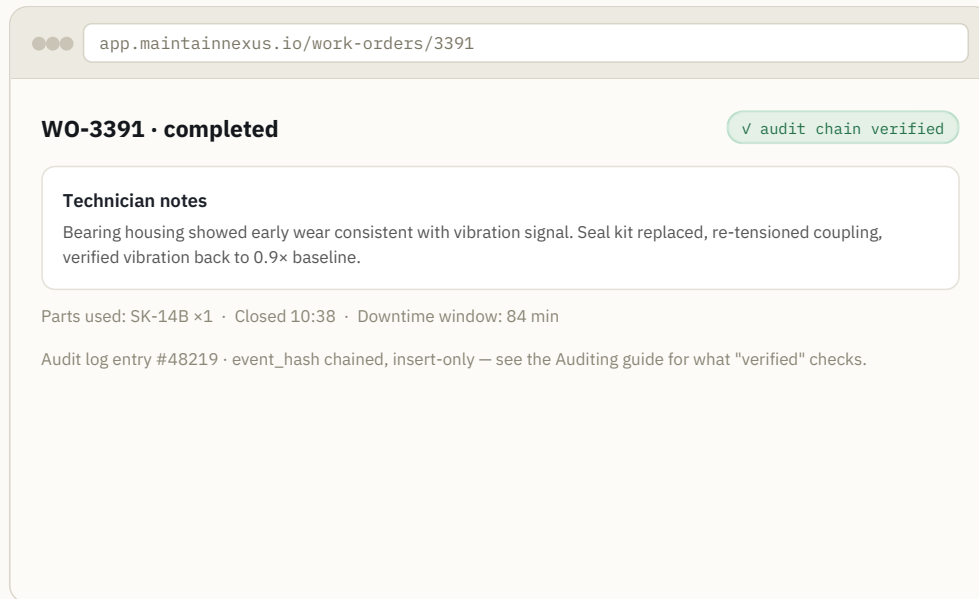
In progress — 3 of 5 steps

updating live over the events stream

This view updates itself — no refresh button, no polling. The same lifecycle events feed the mobile app.

5 Live status

As the technician works through the checklist on mobile (Mockup 4), the same events stream this dashboard subscribes to updates this screen — including the moment their phone was offline and caught up.

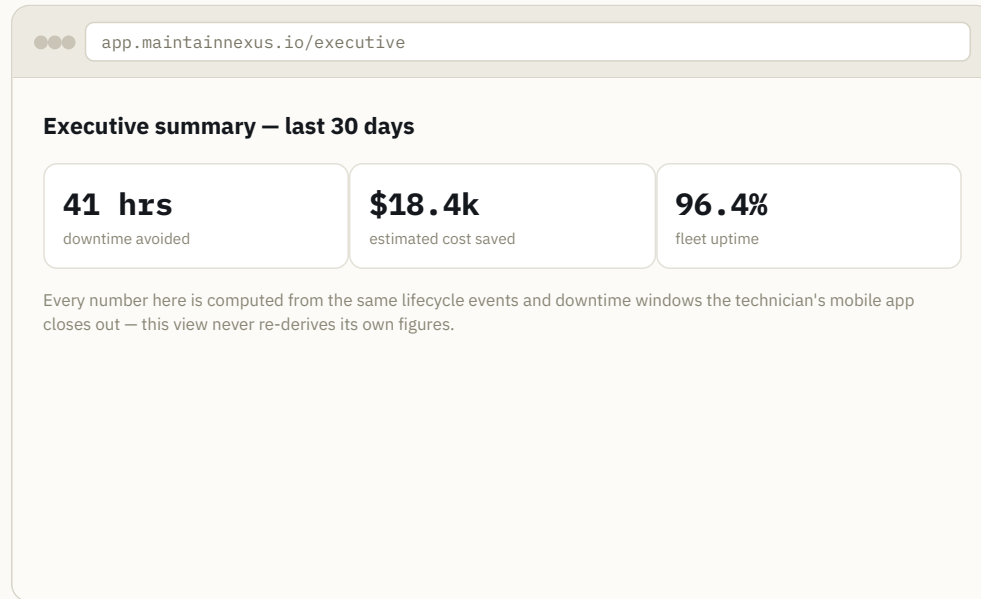


6 Completion & audit trail

The technician's close-out screen (Mockup 5) becomes this record. The chain-verified badge is what §07 Auditing guide calls a standing, automatic check, not a click-to-verify button.

Executive rollup & the integration loop

The flow closes the same way it's described in 00-PROJECT-DOC.md §6: one event log, read by two live clients.

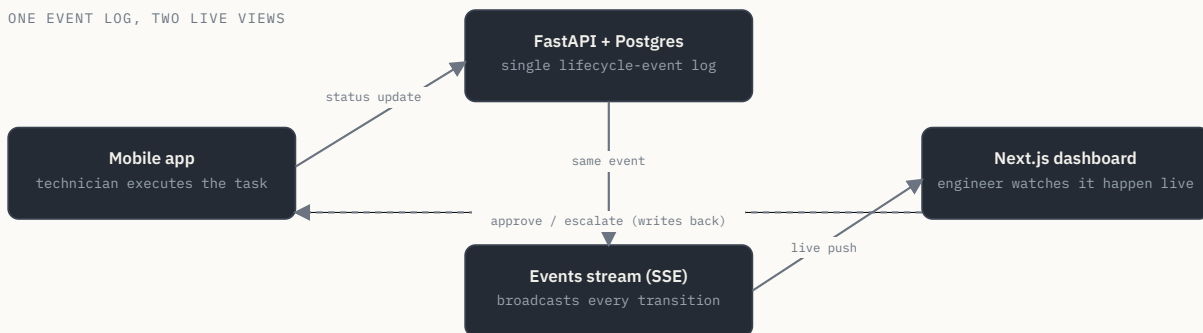


7 Executive summary

Downtime avoided and cost saved roll up from every closed work order like this one — the same figures the pitch deck should pull from, live.

One event log, two live views

ONE EVENT LOG, TWO LIVE VIEWS



Where this fits with the rest of the project

This spec is a companion to the numbered implementation guides — it shows what they describe, it doesn't replace them.

Guide	What it covers that this spec visualizes
08-MOBILE-GUIDE.md	Every screen and journey in Part 1 — API surface, local-store choice, conflict-detection logic.
01-FRONTEND-GUIDE.md	The Next.js screens in Part 2 — role-gated views, live event wiring.
02-BACKEND-GUIDE.md	The lifecycle events, RBAC, and escalation logic both parts render.
03-DATA-ML-GUIDE.md	The risk_score / top_features contract shown in Mockup 3 and Web Screen 3.
06-INTEGRATIONS-GUIDE.md	The SMS fallback in Mockup 8 and the shared auth strategy behind sign-in.
07-AUDITING-GUIDE.md	The chain-verified badges in Web Screens 2 and 6.